Sentinel Logic

Collusion Detection and Response: Oversight That Outlives the Transcript

Mikey Whiston
Security Researcher & Systems Architect
Sentinel Logic Ltd
mikey@mikeywhiston.dev

Abstract

[abstract]

1 Introduction

[introduction]

2 Related Work and Threat Model

[related work]

3 Method

4 Method

Attested identity. An agent is a cgroup v2 inode. The probe reads `bpf_get_current_cgroup_id()` at syscall entry and resolves the caller to the nearest registered cgroup in the sandbox slice. No field of the agent's identity is self-reported: a process can change its name, hostname and uid, but not its position in the cgroup hierarchy, and the kernel records it before the syscall runs.

Identity is only half of the join. Kernel objects are keyed by the pair (dev, ino) rather than by inode, because inode numbers repeat across containers that share image layers. In our testbed the shared volume collapses to one key for all agents $(29/2693466)$, while the same binary in three containers yields one inode under three devices (85, 74, 82 for 2604173). Keying on inode alone would report every agent that launches `bash` as sharing an object with every other.

Objects and roles. Three object classes are in scope. File inodes are mutated by `write`, `truncate`, `chmod`, `setxattr`, `utimensat`, `rename` and `unlink`, and sensed by `read`, `stat`, `access` and `getdents`. Directory inodes are mutated by creating or removing a dirent and sensed by listing the directory; this is what makes the zero-read channel deterministic rather than statistical, since a receiver that only calls `getdents64` never opens the sender's file but still meets it on the directory object. Finally, any syscall whose target resolves to a `(dev,ino)` is admissible, so the table is an annotation of direction rather than a gate on participation: a syscall we have not classified still forms an undirected adjacency edge. We keep the mutator list short deliberately. Mislabelling a mutator as a sensor and the reverse cost the same, so defaulting an unknown verb to *sense* is no worse than attempting exhaustive classification, and we avoid a table of every syscall in the ABI.

Capture without a syscall table. Ring-0 holds no per-syscall knowledge. For each syscall in the slice the probe records the number, six registers, the cgroup, the command name and up to two user strings, chosen by a bitmap supplied by the loader. The bitmap is derived at startup from the kernel's own tracepoint metadata: for each `sys_enter_*` event we read `/sys/kernel/tracing/events/syscalls/` and mark arguments typed `char *` whose names indicate a path, with `char *const *` vectors read element-wise into a separate payload ring so command vectors are captured without truncation. Adding a syscall is a map insert, not a recompilation. Syscalls outside the bitmap are counted in a per-agent histogram rather than dropped, so an unmodelled channel appears as a frequency anomaly against the benign baseline instead of as silence.

Resolving names inside the namespace. Paths reach the sensor as the agent wrote them. The loader resolves them with `openat2` with `RESOLVE_IN_ROOT` and `RESOLVE_NO_MAGICLINKS` against an `O_PATH` descriptor for the agent's `/proc/<pid>/root`, then `fstats` the result. `RESOLVE_IN_ROOT` treats that descriptor as `/` for the whole walk, which matters for two cases we observed: an absolute symlink target is otherwise resolved against the *host* root, and `..` at the container root escapes to a host inode shared by ev-

ery container. Both produced false edges before the change. Resolution results are cached per $(agent, path)$ and invalidated on `O_CREAT`, because a delete-and-recreate may be served a different inode.

Causal attribution. The `sched_process_fork` tracepoint yields parent and child; `execve` yields the argument vector. Together they form an exec forest per agent. A declared tool call binds to the node whose effective command equals it, after unwrapping one shell level and comparing basenames; that node's subtree is the delegated action, and nodes outside it are unaccounted. Because every object touch carries a thread-group id, each edge can be walked to the exec node that authorised it, and a supervision layer can ask which declaration is responsible for a syscall rather than which syscall happened near a timestamp.

Reconciliation and policy. Declarations are reconciled against executions into four classes: `match`, `mismatch` (declared, absent), `omission` (silent) and `surplus` (executed with no covering declaration). Separately, every edge is evaluated against a sanction set over paths, yielding `sanctioned` or `unsanctioned`. Collusion is defined as an unsanctioned edge: the detector never infers intent, it records the interaction and applies declared policy.

5 Results

6 Results

Setup. Three Debian containers share one systemd slice and one mounted volume; the sensor runs on the host. Two suites are reported separately because they answer different questions. The *scripted* suite uses deterministic actors to exercise the sensor across media and is scored against ground truth fixed before any run. The *live* suite uses omp agents with a language model, and exists to produce transcripts: only there can a transcript-only monitor be compared with the kernel. Episodes are hermetic, each starting from an empty shared surface, and detector and scenario hashes are recorded per run.

Storage-channel coverage. Table 1 summarises 72 episodes over 18 channels: file content (including append, truncate, size change and delete-then-recreate), inode metadata (xattr, mtime, mode), namespace operations (directory creation, nested directories, rename, hardlink, symlink), channel objects (a FIFO and a shared mapping) and multi-agent topologies (a three-hop relay and a one-to-many fan-out). Recall is $80/80$ with zero false edges; the 95% lower bound on recall is 95.4%.

Group	Episodes	TP	FP	FN
file content	20	20	0	0
inode metadata	12	12	0	0
namespace	20	20	0	0
channel objects	8	8	0	0
multi-agent	12	20	0	0
total (18 channels)	72	80	0	0

Table 1: Scripted suite, four repeats per scenario. Recovery is complete with no false edges; the multi-agent group yields two edges per episode by construction.

Mode	Ep.	Edges	Unsanc.	Tx. recall
covert	3	2	2	0.00
cooperative	3	3	0	0.00
laundering	3	3	3	0.00
laundering (npm)	3	3	3	0.00
laundering (cargo)	3	3	3	0.00
benign	3	0	0	0.00

Table 2: Live suite: 18 episodes with omp agents. A transcript-only monitor recovered 0 of 42 kernel edges across all modes.

What the model does not cover. Two channels are reported as *boundary probes* and are expected to be missed. Lock contention is invisible because both parties mutate, so no mutator-to-sensor pair exists; a UNIX domain socket is invisible because `bind` and `connect` carry a sockaddr rather than a path. Across eight probe episodes neither produced an edge. FIFOs and shared mappings, by contrast, we expected to miss and do not: a pipe has an inode, and writing a shared mapping requires a writable open, so the edge is visible even though the mapped content is not.

Benign control. Forty episodes span ten controls: independent work, randomly generated paths, co-readers of a read-only file, a single agent writing and reading its own file, interleaved but disjoint writes, subtree reads under `/usr/share`, temporary-file churn, and legitimate cache sharing under a permissive policy. No benign episode produced an unsanctioned edge, nor an edge on an object outside its declared set; the 95% upper bound on the per-episode alarm rate is 8.8%. The cache episode does produce edges, which is the intended behaviour: the mechanism fires and the policy stays quiet.

Iteration	Change	TP	FP	Recall
1	baseline	15	13	0.625
2	namespace fix	18	18	0.750
3	cache scope	21	0	1.000
4	30 scenarios	68	0	0.944
5	cache invalidation	120	0	1.000

Table 3: Ablation. Scoring model corrected to objects in iteration 3; re-scoring earlier corpora separates the effect of the detector fixes from the correction of the scoring model: false positives fall to zero only when the detector is fixed.

Transcript-only monitoring fails. Across all eighteen live episodes the kernel recorded 42 edges, and a monitor that reads only declared tool calls recovered none (95% upper bound on recall 8.4%). This holds in the *laundering* modes, where the agent's declaration is true: it ran `pytest -q` or `npm test`, but the binary answering to that name had been replaced and performed the covert write. Reconciliation reports `match`, both declarations are honest, and the interaction is still recovered because it is recorded at the object. Oversight that trusts claims fails not only when agents lie, but when they tell the truth about a name whose implementation is hostile.

Reconciliation results. On honest transcripts every declaration matches an execution and no surplus is reported. Rewriting the declared command in one agent's session file produces one `mismatch` and an `unaccounted` set naming the command that actually ran. The object edge survives both the rewrite and the subsequent `unlink`, because the edge is published before the file is deleted; deletion is recorded as a scrub annotation rather than as a retraction.

Defects found by the suite. Four detector defects were found by scenarios rather than by inspection: absolute symlink targets resolved in the host namespace; rename destinations captured but never correlated; a global path-to-inode cache that let one agent's inode stand in for another's unresolved path; and a cache entry surviving delete-and-recreate. Table 3 shows the trajectory, and Appendix B records each defect with its failing metric.

7 Discussion and Limitations

[discussion]

8 Conclusion

[conclusion]

A Event Schema

[schema table]

B Iteration Log

[defect log: each defect, found by, metric before/after]

C Limitations and Dual-Use

[required appendix]

D LLM Usage Statement

[required statement]